Electronique Numérique – TP

Par CHENNAOUI Rayan, 2PC-2

Table des matières

INTRODUCTION 3
OBJECTIFS 4
EXERCICE 1 5
EXERCICE 1 : BONUS 9
EXERCICE 2 11
CONCLUSION 12

INTRODUCTION

Ce rapport de travaux pratiques a pour objectif de fournir une prise en main approfondie de l'outil de simulation et de programmation Quartus Prime Lite ainsi que d'explorer l'architecture des FPGA (Field Programmable Gate Array).

Ces exercices pratiques visent à familiariser les étudiants avec la création et la validation de circuits combinatoires au sein d'un FPGA. En travaillant de manière autonome, les étudiants apprendront à créer un projet, à concevoir des schémas de circuits, et à assigner des pins physiques sur une carte FPGA.

Les applications pratiques incluent la réalisation d'un transcodeur binaire pour un afficheur à LED 7-segments et un comparateur à seuils, offrant ainsi une expérience concrète de la programmation et de l'expérimentation avec des dispositifs logiques programmables.

OBJECTIFS

L'objectif principal de ce TP est de nous permettre de nous familiariser avec les concepts fondamentaux de l'électronique numérique en utilisant un outil de simulation et de programmation, Quartus Prime Lite. Les objectifs spécifiques de ce TP sont les suivants :

1. **Prise en main de Quartus Prime Lite :**
   - Apprendre à créer un nouveau projet sous Quartus Prime Lite.
   - Comprendre comment utiliser l'interface utilisateur et les différentes fonctionnalités de Quartus Prime Lite.
   - Nous familiariser avec les fichiers de conception, d'assignation, de simulation et de configuration.

2. **Découverte de l'architecture des FPGA :**
   - Comprendre le fonctionnement et l'architecture des FPGA.
   - Apprendre à configurer et programmer un FPGA.
   - Explorer les capacités des FPGA pour réaliser des circuits logiques programmables.

3. **Réalisation et validation des circuits combinatoires :**
   - Concevoir et simuler des circuits combinatoires simples.
   - Valider le bon fonctionnement des circuits combinatoires en utilisant des méthodes de simulation et de vérification.
   - Expérimenter la programmation des circuits sur une carte FPGA et vérifier leur comportement en conditions réelles.

4. **Applications pratiques :**
   - Réaliser un transcodeur binaire pour un afficheur 7 segments.
   - Utiliser les techniques de synthèse combinatoire pour développer des circuits logiques.
   - Vérifier expérimentalement les circuits conçus en les programmant sur la carte FPGA et en testant leurs sorties.

Ces objectifs visent à renforcer nos compétences pratiques en électronique numérique, en nous préparant à concevoir et à implémenter des circuits logiques complexes dans des dispositifs programmables comme les FPGA.

EXERCICE 1

Pour cet exercice, nous devons réaliser un transcodeur binaire-décimal destiné à un afficheur 7 segments. L'objectif est de convertir une valeur binaire (de 0 à 7) sélectionnée via des interrupteurs en une représentation visuelle sur un afficheur 7 segments. Voici les étapes détaillées de notre démarche :

1. **Élaboration de la Table de Vérité :** Nous avons 3 entrées et 7 sorties. La première étape consiste à établir une table de vérité pour le transcodeur. Chaque entrée binaire (de 0 à 7) doit correspondre à un état des segments de l'afficheur. Les segments a, b, c, d, e, f, et g seront déterminés pour chaque combinaison d'entrée binaire (A, B, C) correspondant à sw2, sw1 et sw0. Nous avons donc établi la table suivante :

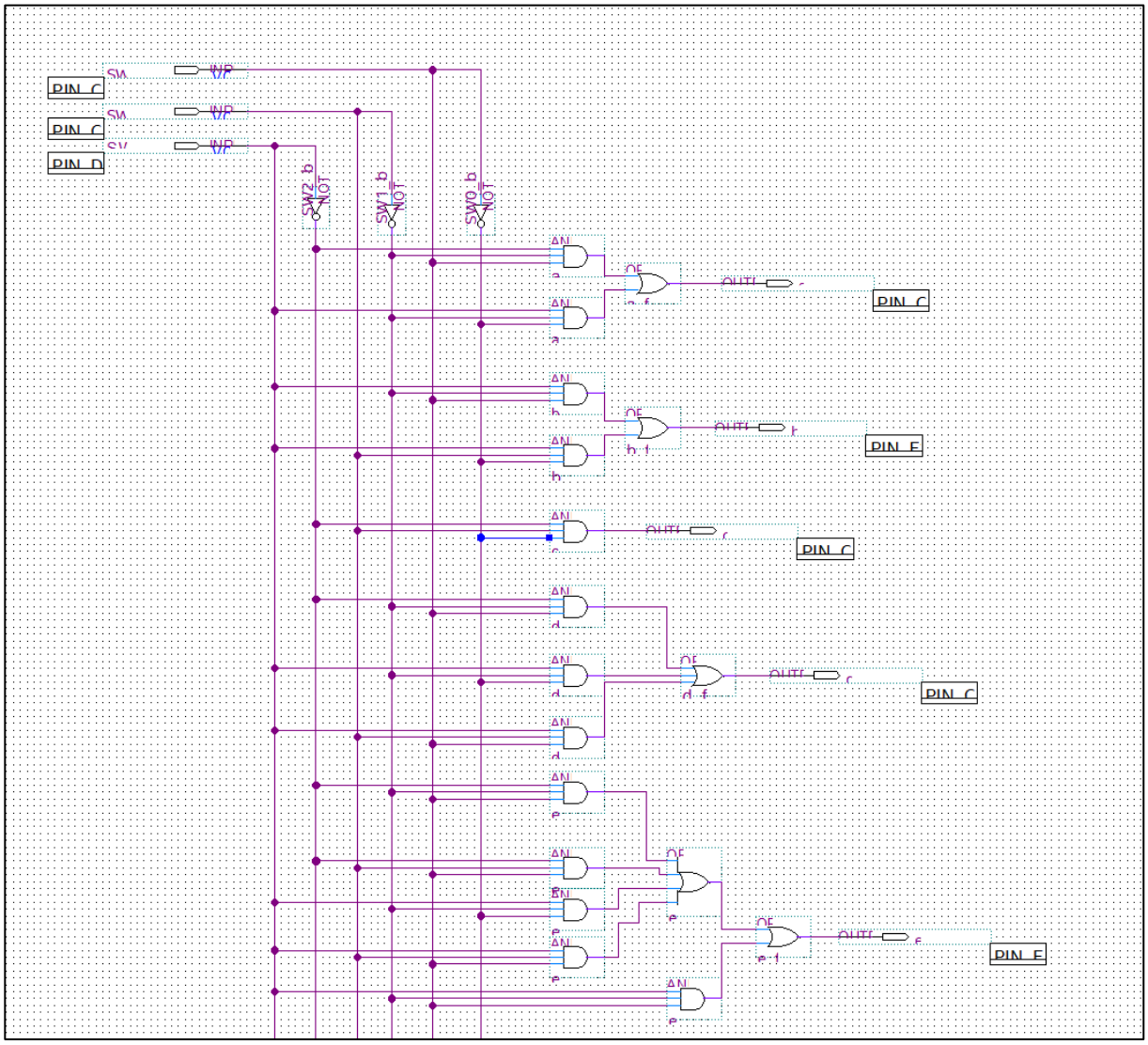
Entrée (ABC)	a	b	c	d	e	f	g
000	0	0	0	0	0	0	1
001	1	0	0	1	1	1	1
010	0	0	1	0	0	1	0
011	0	0	0	0	1	1	0
100	1	0	0	1	1	0	0
101	0	1	0	0	1	0	0
110	0	1	0	0	0	0	0
111	0	0	0	1	1	1	1

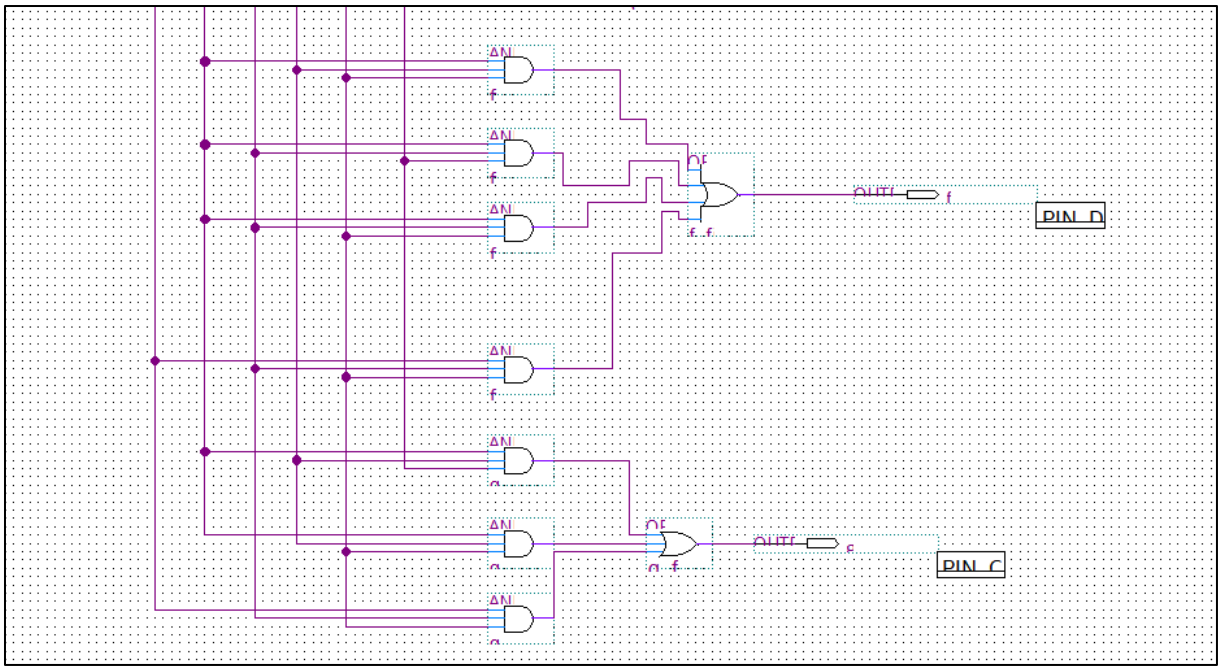
2. **Utilisation des Tableaux de Karnaugh :** Ensuite, pour simplifier les équations logiques pour chaque segment, nous avons utilisé les tableaux de Karnaugh. Ces tableaux permettent de minimiser les fonctions logiques en regroupant les '1' adjacents. Nous avons ainsi déterminé les équations minimisées pour chaque segment a, b, c, d, e, f, et g.

   Nous avons alors déterminé toutes les équations :

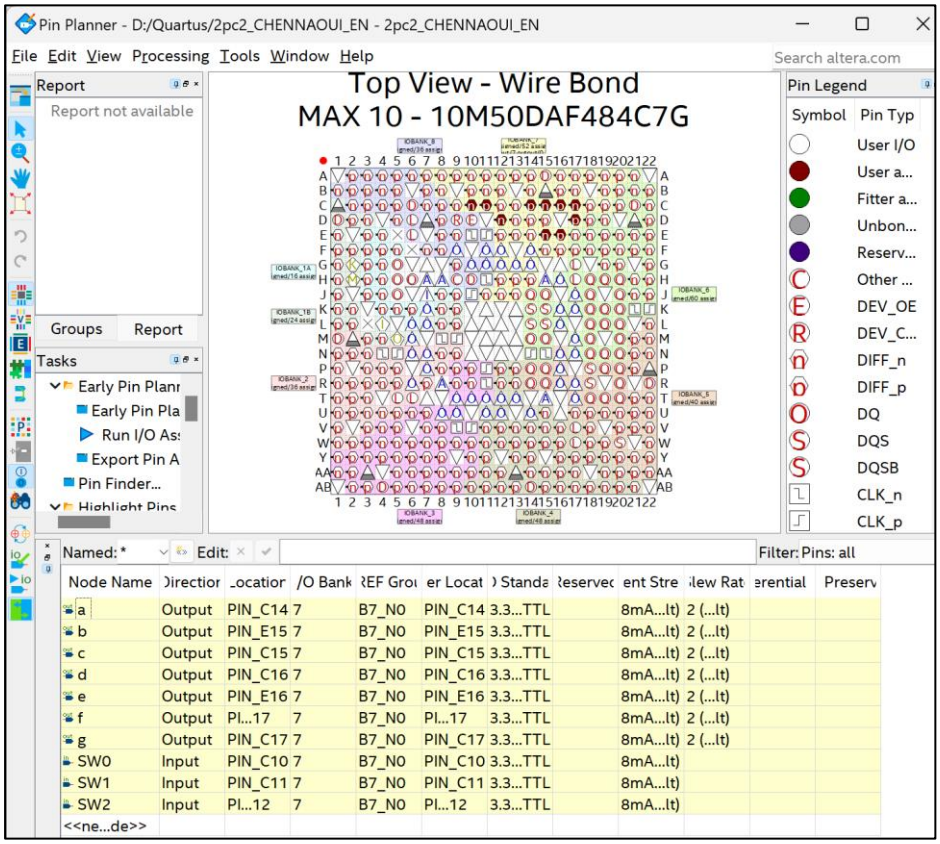
a : $\overline{sw_2} * \overline{sw_1} * sw_0 + sw_2 * \overline{sw_1} * \overline{sw_0}$
b : $sw_2 * \overline{sw_1} * sw_0 + sw_2 * sw_1 * \overline{sw_0}$
c : $\overline{sw_2} * sw_1 * \overline{sw_0}$
d : $\overline{sw_2} * \overline{sw_1} * sw_0 + sw_2 * \overline{sw_1} * \overline{sw_0} + SW_2 * sw_1 * sw_0$
e : $\overline{sw_2} * \overline{sw_1} * sw_0 + \overline{sw_2} * sw_1 * sw_0 + sw_2 * \overline{sw_1} * \overline{sw_0} + sw_2 * \overline{sw_1} * sw_0 + sw_2 * sw_1 * sw_0$
f : $\overline{sw_2} * \overline{sw_1} * sw_0 + \overline{sw_2} * sw_1 * \overline{sw_0}$
g : $\overline{sw_2} * \overline{sw_1} * \overline{sw_0} + \overline{sw_2} * \overline{sw_1} * sw_0 + sw_2 * sw_1 * sw_0$

3. **Conception du Schéma :** Une fois les équations logiques simplifiées obtenues, nous avons créé un nouveau schéma dans Quartus Prime Lite. En utilisant les portes logiques de base disponibles dans la bibliothèque, nous avons implémenté chaque équation pour chaque segment. Le schéma final relie les entrées binaires (A, B, C) aux segments de l'afficheur.

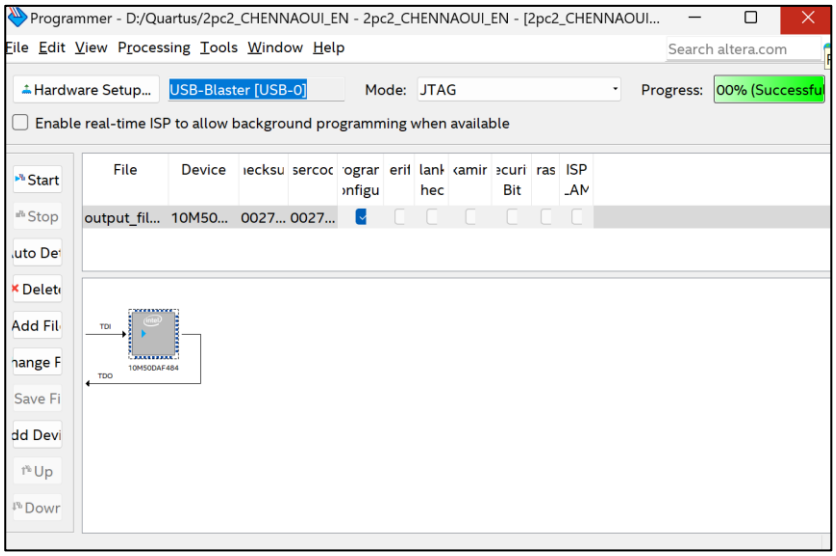




4. **Assignation des Pins :** Les signaux ont ensuite été assignés aux entrées et sorties de la carte FPGA. Les entrées A, B, et C correspondent aux interrupteurs SW0, SW1, et SW2, respectivement. Les sorties des segments (a à g) sont connectées aux pins correspondants de l'afficheur 7 segments de la carte FPGA. En lançant Pin Planner on peut assigner un nom à chaque pin, ainsi que l'assigner à une Location :



5. **Vérification Expérimentale :** Enfin, nous avons compilé le projet et chargé le fichier de programmation sur la carte FPGA. En manipulant les interrupteurs, nous avons vérifié que chaque combinaison binaire s'affiche correctement sur l'afficheur 7 segments selon la table de vérité établie. En lançant Programmer, et en appuyant sur Start :



Quelques exemples :

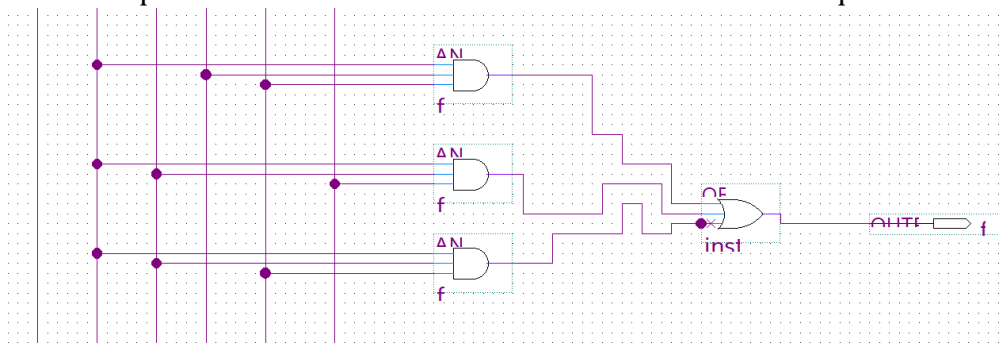
EXERCICE 1 : BONUS

Pour cet exercice bonus, nous avons pour objectif de modifier l'affichage du chiffre 7 sur l'afficheur 7 segments. Initialement, le chiffre 7 est représenté en allumant les segments a, b, et c, laissant les segments d, e, f, et g éteints. Nous voulons ajouter une spécificité : allumer également le segment en haut à gauche (segment f) pour le chiffre 7.

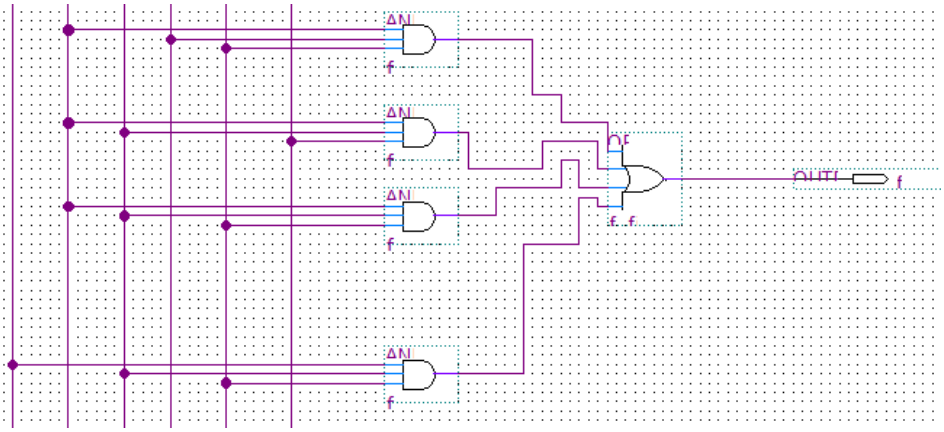
La première étape consiste à modifier la table de vérité pour refléter ce changement. Pour l'entrée binaire correspondant au chiffre 7 (111), nous allons allumer le segment f en plus des segments a, b, et c. Voici la table de vérité modifiée :

Entrée (ABC)	a	b	c	d	e	f	g
000	0	0	0	0	0	0	1
001	1	0	0	1	1	1	1
010	0	0	1	0	0	1	0
011	0	0	0	0	1	1	0
100	1	0	0	1	1	0	0
101	0	1	0	0	1	0	0
110	0	1	0	0	0	0	0
111	0	0	0	1	1	0	1

Ceci se traduit par la modification suivante dans le schéma électrique :



Comparé au schéma précédent pour f :



On obtient finalement un 7 modifié :

EXERCICE 2

J'ai fait rapidement quelques recherches pour élaborer des éléments de réflexion sur la conception d'un comparateur à seuils, comme spécifié dans l'énoncé.

Pour concevoir un circuit logique qui donne une sortie S haute quand le nombre binaire sur 4 bits A=$A3$ $A2$ $A1$ $A0$ est supérieur à 8 et inférieur à 13, nous devons analyser les valeurs de A et identifier les conditions où S doit être haute.

Les valeurs binaires pour lesquelles S doit être haute sont :

- 9 (1001)
- 10 (1010)
- 11 (1011)
- 12 (1100)

La table de vérité correspondante est :

A3	***A2***	***A1***	***A0***	***S***
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1

À partir de cette table, nous utiliserions des tableaux de Karnaugh pour simplifier l'équation logique de S. Ensuite, nous programmerions l'architecture du comparateur de seuils à l'aide des portes logiques disponibles dans la bibliothèque de Quartus Prime Lite. Pour les signaux, nous assignerions les entrées et sorties comme suit :

- $A0$: (SW0)
- $A1$: (SW1)
- $A2$: (SW2)
- $A3$: (SW3)
- S : (LEDR3)

Enfin, nous compilerions le projet et programmerions la carte FPGA. En manipulant les interrupteurs correspondant aux bits $A0A0$, $A1A1$, $A2A2$, et $A3A3$, nous pourrions observer la sortie SS sur la LEDR3 et vérifier que SS est haute uniquement pour les valeurs de AA comprises entre 9 et 12.

CONCLUSION

Ce travail pratique a permis aux étudiants de développer des compétences essentielles dans l'utilisation de Quartus Prime Lite et la programmation des FPGA. En passant par les étapes de création de projet, de conception de schémas, et de programmation physique des circuits, les étudiants ont acquis une compréhension approfondie des circuits combinatoires et de leur implémentation dans des FPGA.

Les exercices appliqués, comme le transcodeur binaire et le comparateur à seuils, ont consolidé cette compréhension en offrant des exemples concrets de synthèse combinatoire et de vérification expérimentale. Ces compétences sont fondamentales pour toute future application dans le domaine de l'électronique numérique et des systèmes programmables.